Debounce & Closures

JavaScript quick-reader | Your 4 questions, answered without extra theory

Q1. Why write search = debounce(callback, 500)?

Because debounce() creates a function once, and search() reuses that same function and its timer. Calling debounce() again for every keystroke would create separate timers that cannot cancel each other.

Q2. Is timer a static variable, declared once?

One timer variable is created per call to debounce(). It survives because the returned function closes over it. It is not a JavaScript static variable; the engine manages its storage. A second debounce() call gets a different timer variable.

Q3. Does the closure "bag theory" apply to timer and delay?

Yes. The returned function keeps access to timer, delay, and callback from its surrounding scope. Think of a "bag" of accessible variables, but they are live bindings, not copied snapshots.

Q4. When should I use a closure / function inside a function?

Ask: Does this function need to remember a value after the outer function finishes? If yes, a closure is useful. Examples: debounce remembers its timer; a counter remembers its count; a wrapper remembers the original callback.

The example that connects all 4 answers

```
function debounce(callback, delay) {
  let timer;
  return function (text) {
    clearTimeout(timer);
    timer = setTimeout(() => callback(text), delay);
  };
}

const search = debounce((text) => console.log(text), 500);
search("V");
search("Vi");  // Only "Vi" runs after 500 ms of inactivity.
```

Remember: One `debounce()` call creates one private timer. Each `search()` call resets it.

Conversation reference

Open ChatGPT conversation history | Direct share URL for this specific chat: not provided yet.
To make this link open this exact conversation, replace it with the URL from Share > Copy link.